스프링 부트 3 살펴보기

Spring Boot = P.L + Build Tool + Spring Framework

I'm honeymon.

```java
public class Honeymon {
  private static final String NAME = "Honeymon(김지헌)";
  private static final String EMAIL = "ihoneymon@gmail.com";
  private static final String COMPANY = "Kurly";
  private static final String DESCRIPTION = """
      Spring Boot 3.0 출시에 맞춰
      - Java
      - 스프링 프레임워크 6
      - 스프링 부트 3
      - 정리
      의 순서로 설명을 진행하겠습니다.
      """;
}
```

https://honeymon.io/

Spring Boot 3.0 요구사항

- Java 17(Java 19 까지 호환)
- Jakarta EE 10
- Spring Framework 6.0 이상
- 빌드도구
  - Gradle 7.X(7.5 이상)
  - Maven3(3.5+)

https://docs.spring.io/spring-boot/docs/3.0.0-RC2/reference/htmlsingle/#getting-started.system-requirements

Java

Java 10 - Local Variable Type Inference

변수명은 유용하게(의미있게)

```
// ORIGINAL
List<Customer> x = dbconn.executeQuery(query);

// GOOD
var custList = dbconn.executeQuery(query);
```

https://openjdk.org/projects/amber/guides/lvti-style-guide

Java 17

Version	기능
10	Local Variable Type Inference: var • Local Variable Type Inference Style Guide
14	Switch Expression
15	Text Block • Programmer's Guide to Text Blocks
16	Records Pattern Matching for instanceof
17	Sealed Class

https://docs.oracle.com/en/java/javase/17/language/java-language-changes.html

Java 10 - Local Variable Type Inference

지역변수 범위는 최소화(Effective Java 3th: Item 57)

```
var items = new ArrayList<Item>(...);
items.add(MUST_BE_PROCESSED_LAST);
for (var item : items) ...
```

```
var items = new HashSet<Item>(...);
items.add(MUST_BE_PROCESSED_LAST);
for (var item : items) ...
```

https://openjdk.org/projects/amber/guides/lvti-style-guide

Java 10 - Local Variable Type Inference

지역변수 범위는 최소화(Effective Java 3th: Item 57)

```
var items = new ArrayList<Item>(...);
items.add(MUST_BE_PROCESSED_LAST);
for (var item : items) ...
```

```
var items = new HashSet<Item>(...);

// ... 100 lines of code ...

items.add(MUST_BE_PROCESSED_LAST);
for (var item : items) ...
```

https://openjdk.org/projects/amber/guides/lvti-style-guide

Java 10 - Local Variable Type Inference

초기화에서 충분한 정보를 제공하는 경우!

```
// ORIGINAL
ByteArrayOutputStream outputStream = new ByteArrayOutputStream();

// GOOD
var outputStream = new ByteArrayOutputStream();
```

```
// ORIGINAL
BufferedReader reader = Files.newBufferedReader(...);
List<String> stringList = List.of("a", "b", "c");

// GOOD
var reader = Files.newBufferedReader(...);
var stringList = List.of("a", "b", "c");
```

https://openjdk.org/projects/amber/guides/lvti-style-guide

Java 10 - Local Variable Type Inference

연속되거나 내재화된 표현을 분리할 때

```
return strings.stream()
              .collect(groupingBy(s -> s, counting()))
              .entrySet()
              .stream()
              .max(Map.Entry.comparingByValue())
              .map(Map.Entry::getKey);
```

문자열 목록에서 문자별로 묶고 갯수 확인

갯수를 확인한 문자열 중에 가장 갯수가 많은 키값 확인

https://openjdk.org/projects/amber/guides/lvti-style-guide

Java 10 - Local Variable Type Inference

```
Map<String, Long> freqMap = strings.stream()
                                  .collect(groupingBy(s -> s, counting()));
Optional<Map.Entry<String, Long>> maxEntryOpt = freqMap.entrySet()
                                                       .stream()
                                                       .max(Map.Entry.comparingByValue());
return maxEntryOpt.map(Map.Entry::getKey);
```

```
var freqMap = strings.stream()
                     .collect(groupingBy(s -> s, counting()));
var maxEntryOpt = freqMap.entrySet()
                         .stream()
                         .max(Map.Entry.comparingByValue());
return maxEntryOpt.map(Map.Entry::getKey);
```

https://openjdk.org/projects/amber/guides/lvti-style-guide

Java 10 - Local Variable Type Inference

인터페이스 프로그래밍에 대한 걱정은 하지마세요.

```
// ORIGINAL
List<String> list = new ArrayList<>();
```

```
// Inferred type of list is ArrayList<String>
var list = new ArrayList<String>();
```

https://openjdk.org/projects/amber/guides/lvti-style-guide

Java 10 - Local Variable Type Inference

다이아몬드 연산 혹은 제네릭 메서드 사용은 조심!

```
PriorityQueue<Item> itemQueue = new PriorityQueue<Item>();
```

```
// OK: both declare variables of type PriorityQueue<Item>
PriorityQueue<Item> itemQueue = new PriorityQueue<>();
var itemQueue = new PriorityQueue<Item>();
```

```
// DANGEROUS: infers as PriorityQueue<Object>
var itemQueue = new PriorityQueue<>();
```

https://openjdk.org/projects/amber/guides/lvti-style-guide

Java 10 - Local Variable Type Inference

다이아몬드 연산 혹은 제네릭 메서드 사용은 조심!

```
// DANGEROUS: infers as List<Object>
var list = List.of();
```

```
// OK: itemQueue infers as PriorityQueue<String>
Comparator<String> comp = ... ;
var itemQueue = new PriorityQueue<>(comp);
```

```
// OK: infers as List<BigInteger>
var list = List.of(BigInteger.ZERO);
```

https://openjdk.org/projects/amber/guides/lvti-style-guide

Java 14 - Switch Expression

```java
public enum Day { SUNDAY, MONDAY, TUESDAY,
    WEDNESDAY, THURSDAY, FRIDAY, SATURDAY; }

// ...

    int numLetters = 0;
    Day day = Day.WEDNESDAY;
    switch (day) {
        case MONDAY:
        case FRIDAY:
        case SUNDAY:
            numLetters = 6;
            break;
        case TUESDAY:
            numLetters = 7;
            break;
        case THURSDAY:
        case SATURDAY:
            numLetters = 8;
            break;
        case WEDNESDAY:
            numLetters = 9;
            break;
        default:
            throw new IllegalStateException("Invalid day: " + day);
    }
    System.out.println(numLetters);
```

https://docs.oracle.com/en/java/javase/17/language/switch-expressions.html

Java 14 - Switch Expression

'Case L:' 레이블

```
Day day = Day.WEDNESDAY;
System.out.println(
    switch (day) {
        case MONDAY, FRIDAY, SUNDAY -> 6;
        case TUESDAY                -> 7;
        case THURSDAY, SATURDAY     -> 8;
        case WEDNESDAY              -> 9;
        default -> throw new IllegalStateException("Invalid day: " + day);
    }
);
```

https://docs.oracle.com/en/java/javase/17/language/switch-expressions.html

Java 14 - Switch Expression

'Case L -> ' Label

```
int numLetters = 0;
Day day = Day.WEDNESDAY;
switch (day) {
    case MONDAY, FRIDAY, SUNDAY -> numLetters = 6;
    case TUESDAY                -> numLetters = 7;
    case THURSDAY, SATURDAY     -> numLetters = 8;
    case WEDNESDAY              -> numLetters = 9;
    default -> throw new IllegalStateException("Invalid day: " + day);
};
System.out.println(numLetters);
```

https://docs.oracle.com/en/java/javase/17/language/switch-expressions.html

Java 14 - Switch Expression

'Case L' & `yield`

```
Day day = Day.WEDNESDAY;
int numLetters = switch (day) {
    case MONDAY:
    case FRIDAY:
    case SUNDAY:
        System.out.println(6);
        yield 6;
    case TUESDAY:
        System.out.println(7);
        yield 7;
    case THURSDAY:
    case SATURDAY:
        System.out.println(8);
        yield 8;
    case WEDNESDAY:
        System.out.println(9);
        yield 9;
    default:
        throw new IllegalStateException("Invalid day: " + day);
};
System.out.println(numLetters);
```

https://docs.oracle.com/en/java/javase/17/language/switch-expressions.html

Java 14 - Switch Expression

'Case L' & `yield`

```
int numLetters = switch (day) {
    case MONDAY, FRIDAY, SUNDAY -> {
        System.out.println(6);
        yield 6;
    }
    case TUESDAY -> {
        System.out.println(7);
        yield 7;
    }
    case THURSDAY, SATURDAY -> {
        System.out.println(8);
        yield 8;
    }
    case WEDNESDAY -> {
        System.out.println(9);
        yield 9;
    }
    default -> {
        throw new IllegalStateException("Invalid day: " + day);
    }
};
```

https://docs.oracle.com/en/java/javase/17/language/switch-expressions.html

Java 14 - Switch Expression

```
int i = switch (day) {
    case MONDAY -> {
        System.out.println("Monday");
        // ERROR! Block doesn't contain a yield statement
    }
    default -> 1;
};
```

```
i = switch (day) {
    case MONDAY, TUESDAY, WEDNESDAY:
        yield 0;
    default:
        System.out.println("Second half of the week");
        // ERROR! Group doesn't contain a yield statement
};
```

https://docs.oracle.com/en/java/javase/17/language/switch-expressions.html

Java 14 - Switch Expression

```
for (int i = 0; i < MAX_VALUE; ++i) {
    int k = switch (e) {
        case 0:
            yield 1;
        case 1:
            yield 2;
        default:
            continue z;
            // ERROR! Illegal jump through a switch expression
    };
// ...
}
```

https://docs.oracle.com/en/java/javase/17/language/switch-expressions.html

Java 15 - Text Block

```
// Using a literal string
String dqName = "Pat Q. Smith";

// Using a text block
String tbName = """
                Pat Q. Smith""";
```

```
// Both dqName and tbName are strings of equal value
dqName.equals(tbName)    // true

// Both dqName and tbName intern to the same string
dqName == tbName         // true
```

https://docs.oracle.com/en/java/javase/14/text-blocks/index.html

Java 15 - Text Block

```
String str = "The old";
String tb = """
            the new""";
String together = str + " and " + tb + ".";
```

```
System.out.println("""
    This is the first line
    This is the second line
    This is the third line
    """);
```

```
"""
John Q. Smith""".substring(8).equals("Smith")    // true
```

https://docs.oracle.com/en/java/javase/17/language/java-language-changes.html

Java 15 - Text Block

```
// ORIGINAL
String message = "'The time has come,' the Walrus said,\n" +
                 "'To talk of many things:\n" +
                 "Of shoes -- and ships -- and sealing-wax --\n" +
                 "Of cabbages -- and kings --\n" +
                 "And why the sea is boiling hot --\n" +
                 "And whether pigs have wings.'\n";
```

```
// BETTER
String message = """
    'The time has come,' the Walrus said,
    'To talk of many things:
    Of shoes -- and ships -- and sealing-wax --
    Of cabbages -- and kings --
    And why the sea is boiling hot --
    And whether pigs have wings.'
    """;
```

https://docs.oracle.com/en/java/javase/17/language/java-language-changes.html

Java 15 - Text Block

```
// ERROR
String name = """Pat Q. Smith""";

// ERROR
String name = """red
                 green
                 blue
                 """;

// OK
String name = """
    red
    green
    blue
    """;
```

```
String name = "red\n" +
              "green\n" +
              "blue\n";
```

https://docs.oracle.com/en/java/javase/17/language/java-language-changes.html

Java 15 - Text Block

마지막 텍스트 블록 닫는 위치에 따라서 줄바꿈(\n) 추가가 결정됨

```java
@Test
void testBlockEquivalent() {
    var name = """
            red
            green
            blue"""; //마지막 줄 '\n' 빼고 싶다면!! blue"""
    var name2 = "red\ngreen\nblue";

    assertThat(name2).isEqualTo(name);
}
```

```java
@Test
void testJsonBlockEquivalent() {
    var jsonLiteralBlock = """
            {
                "name":"Honeymon"
            }
            """;
    var jsonLiteralBlock2 = "{\n" +
            "    \"name\":\"Honeymon\"\n" +
            "}\n";

    assertThat(jsonLiteralBlock2).isEqualTo(jsonLiteralBlock);
}
```

https://docs.oracle.com/en/java/javase/17/language/java-language-changes.html

Java 16 - Records

레코드(record) 선언시

- 접근자
- 생성자
- equals
- hashCode
- toString

순수한 데이터 이송(Data carrier) 역할 의도

final 필드

```
record Rectangle(double length, double width) { }
```

```
public final class Rectangle {
    private final double length;
    private final double width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    double length() { return this.length; }
    double width()  { return this.width; }

    // Implementation of equals() and hashCode(), which specify
    // that two record objects are equal if they
    // are of the same type and contain equal field values.
    public boolean equals...
    public int hashCode...

    // An implementation of toString() that returns a string
    // representation of all the record class's fields,
    // including their names.
    public String toString() {...}
}
```

https://docs.oracle.com/en/java/javase/17/language/records.html

Java 16 - Records

레코드 클래스 정식 생성자(Canonical Constructor)

: 필드 초기화 필요함

```
record Rectangle(double length, double width) {
    public Rectangle(double length, double width) {
        if (length <= 0 || width <= 0) {
            throw new java.lang.IllegalArgumentException(
                String.format("Invalid dimensions: %f, %f", length, width));
        }
        this.length = length;
        this.width = width;
    }
}
```

https://docs.oracle.com/en/java/javase/17/language/records.html

Java 16 - Records

레코드 클래스 압축 생성자(compact Constructor)

: 필드 초기화 필요없음

```
record Rectangle(double length, double width) {
    public Rectangle {
        if (length <= 0 || width <= 0) {
            throw new java.lang.IllegalArgumentException(
                String.format("Invalid dimensions: %f, %f", length, width));
        }
    }
}
```

압축 생성자가 종결되는 시점에서
필드할당

https://docs.oracle.com/en/java/javase/17/language/records.html

Java 16 - Records

레코드 클래스 압축 생성자(compact Constructor)

: 필드 초기화 필요없음

```
record Rectangle(double length, double width) {
    public Rectangle {
        if (length <= 0 || width <= 0) {
            throw new java.lang.IllegalArgumentException(
                String.format("Invalid dimensions: %f, %f", length, width));
        }
    }
}
```

압축 생성자가 종결되는 시점에서
필드할당

https://docs.oracle.com/en/java/javase/17/language/records.html

Java 16 - Records

정적(static) 필드, 정적 초기화, 정적 메서드 사용가능

```
record Rectangle(double length, double width) {

    // Static field
    static double goldenRatio;

    // Static initializer
    static {
        goldenRatio = (1 + Math.sqrt(5)) / 2;
    }

    // Static method
    public static Rectangle createGoldenRectangle(double width) {
        return new Rectangle(width, width * goldenRatio);
    }
}
```

https://docs.oracle.com/en/java/javase/17/language/records.html

Java 16 - Records

인스턴스 초기화는 허용되지 않음

```
record Rectangle(double length, double width) {

    // Field declarations must be static:
    BiFunction<Double, Double, Double> diagonal;

    // Instance initializers are not allowed in records:
    {
        diagonal = (x, y) -> Math.sqrt(x*x + y*y);
    }
}
```

https://docs.oracle.com/en/java/javase/17/language/records.html

Java 16 - Records

Jackson 직렬화(Serialization)는 2.12.3 자동지원, 이전에는...

```
@Test
void serialization() throws JsonProcessingException {
  ObjectMapper objectMapper = new ObjectMapper();

  var rectangleRecord1 =
      new RectangleRecord(BigDecimal.valueOf(100.0), BigDecimal.valueOf(200.0));

  assertThat(objectMapper.writeValueAsString(rectangleRecord1))
      .isEqualTo( expected: "{\"width\":100.0,\"height\":200.0}");

  var source = "{\"width\":100.0,\"height\":200.0}";
  assertThat(objectMapper.readValue(source, RectangleRecord.class)).isEqualTo(rectangleRecord1);
}
```

https://dev.to/brunooliveira/practical-java-16-using-jackson-to-serialize-records-4og4

Java 16 - Pattern Matching for instanceof

class casting 을 줄일 수 있게 되었음

```java
public static double getPerimeter(Shape shape) throws IllegalArgumentException {
    if (shape instanceof Rectangle) {
        Rectangle r = (Rectangle) shape;
        return 2 * r.length() + 2 * r.width();
    } else if (shape instanceof Circle) {
        Circle c = (Circle) shape;
        return 2 * c.radius() * Math.PI;
    } else {
        throw new IllegalArgumentException("Unrecognized shape");
    }
}
```

```java
public static double getPerimeter(Shape shape) throws IllegalArgumentException {
    if (shape instanceof Rectangle r) {
        return 2 * r.length() + 2 * r.width();
    } else if (shape instanceof Circle c) {
        return 2 * c.radius() * Math.PI;
    } else {
        throw new IllegalArgumentException("Unrecognized shape");
    }
}
```

https://docs.oracle.com/en/java/javase/16/language/pattern-matching-instanceof-operator.html

Java 16 - Pattern Matching for instanceof

패턴 변수 범위는 instanceof 가 true 인 경우 적용

```
public static double getPerimeter(Shape shape) throws IllegalArgumentException {
    if (shape instanceof Rectangle s) {
        // You can use the pattern variable s (of type Rectangle) here.
    } else if (shape instanceof Circle s) {
        // You can use the pattern variable s of type Circle here
        // but not the pattern variable s of type Rectangle.
    } else {
        // You cannot use either pattern variable here.
    }
}
```

```
public static boolean bigEnoughRect(Shape s) {
    if (!(s instanceof Rectangle r)) {
        // You cannot use the pattern variable r here because
        // the predicate s instanceof Rectangle is false.
        return false;
    }
    // You can use r here.
    return r.length() > 5;
}
```

https://docs.oracle.com/en/java/javase/16/language/pattern-matching-instanceof-operator.html

Java 16 - Pattern Matching for instanceof

패턴 변수 범위는 instanceof 가 true 인 경우 적용

```
if (shape instanceof Rectangle r && r.length() > 5) {
    // ...
}
```

```
if (shape instanceof Rectangle r || r.length() > 0) { // error
    // ...
}
```

https://docs.oracle.com/en/java/javase/16/language/pattern-matching-instanceof-operator.html

Java 17 - Sealed Class

이름 그대로 '봉인된(Sealed) 클래스': 사용할 수 있는(같은 모듈 혹은 패키지에 있는) 클래스를 지정한다.

```
public sealed class Shape
    permits Circle, Square, Rectangle {
}
```

```
public final class Circle extends Shape {
    public float radius;
}
```

https://docs.oracle.com/en/java/javase/17/language/sealed-classes-and-interfaces.html

Java 17 - Sealed Class

```
public non-sealed class Square extends Shape {
   public double side;
}
```

```
public sealed class Rectangle extends Shape permits FilledRectangle {
    public double length, width;
}
```

```
public final class FilledRectangle extends Rectangle {
    public int red, green, blue;
}
```

https://docs.oracle.com/en/java/javase/17/language/sealed-classes-and-interfaces.html

Spring Framework 6

Spring Framework 6.0

- Baseline
  - Java 17
  - Jakarta EE 10(Jakarta EE 9 지원: 네임스페이스 변경(javax -> jakarta)
- General Core
  - AOT(Ahead Of Time) 을 통한 실행속도 향상
  - PathMatchingResourcePatternResource 사용 클래스패스 경로 자원 검색
  - DefaultFormattingConversionService - ISO 기반 java.time 유형 파싱
- Data Accessing and Transaction
  - JPA support for Hibernate ORM 6.1
  - Upgrade to R2DBC 1.0(R2DBC 트랜잭션 정의 포함)

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x/

Spring Framework 6.0

- Spring Messaging
  - RSocket interface client - @RSocketExchange 서비스 인터페이스
  - Netty 5 기반 Reactor Netty 2 지원
  - Jakarta WebSocket 2.1 및 표준 WebSocket 프로토콜 업그레이드 기작 지원
- General Web
  - HTTP interface client (@HttpExchange) 서비스 인터페이스
  - RFC 7807 - Problem Detail
  - HTTP status code 제어
  - Jackson 2.14 Support
  - Servlet 6.0 맞춤
- Spring MVC
  - PathPatternParser 기본 사용
  - Tiles 와 FreeMarker JSP 지원 제거

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x/

Spring Framework 6.0

- Spring WebFlux
  - PartEvent - Stream multipart form upload
  - ResponseEntityExceptionHandler - WebFlux exception
  - Flux - non-streaming media types 반환
  - Netty 5 기반 Reactor Netty 2 지원
  - JDK HttpClient 를 통합한 WebClient
- Observability
  - Micrometer Observation 를 이용한 직접 관측 연결
- Testing
  - JVM 혹은 GraalVM 상에서 AOT 처리된 애플리케이션 컨텍스트 테스트 지원
  - HtmlUnit 2.64 요청 파라미터 제어 통합
  - Servlet API 6.0 을 기반한 서블릿 Mock(MockHttpServletRequest, MockHttpSession) 지원

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x/

Spring Framework 6.0 - Baseline

Jakarta EE 9 Release plan!

- 네임스페이스 변경: javax -> jakarta
  - 런타임 공급업체가 새로운 네임스페이스에 맞춰 개발을 진행
  - 오래된 Java EE 8 API 제거
  - 새로운 밴더가 Jakarta EE 생태계에 진입할 수 있도록 지원
- Jakarta EE 9 에 맞춰 사양문서(Specification Documentation) 갱신
- Java EE 8 에 대한 하위호환성 보장안됨
  - 대신 이전(migration)할 수 있는 도구 지원
- Java SE 8 소스 컴파일 지원

https://eclipse-ee4j.github.io/jakartaee-platform/jakartaee9/JakartaEE9ReleasePlan

Spring Framework 6.0 - Baseline

Jakarta EE 10(Tomcat 10.0.X 지원종료로 Jakarta EE 9 에서 변경, Java 11 최소)

Java enterprise platform history

Platform version	Released	Specification	Java SE Support	Important Changes
Jakarta EE 10	2022-09-13[9]	10	Java SE 17 Java SE 11	Removal of deprecated items in Servlet, Faces, CDI and EJB (Entity Beans and Embeddable Container). CDI-Build Time.
Jakarta EE 9.1	2021-05-25[10]	9.1	Java SE 11 Java SE 8	JDK 11 support
Jakarta EE 9	2020-12-08[11]	9	Java SE 8	API namespace move from `javax` to `jakarta`
Jakarta EE 8	2019-09-10[12]	8	Java SE 8	Full compatibility with Java EE 8
Java EE 8	2017-08-31	JSR 366	Java SE 8	HTTP/2 and CDI based Security
Java EE 7	2013-05-28	JSR 342	Java SE 7	WebSocket, JSON and HTML5 support
Java EE 6	2009-12-10	JSR 316	Java SE 6	CDI managed Beans and REST
Java EE 5	2006-05-11	JSR 244	Java SE 5	Java annotations
J2EE 1.4	2003-11-11	JSR 151	J2SE 1.4	WS-I interoperable web services[13]
J2EE 1.3	2001-09-24	JSR 58	J2SE 1.3	Java connector architecture[14]
J2EE 1.2	1999-12-17	1.2	J2SE 1.2	Initial specification release

https://en.wikipedia.org/wiki/Jakarta_EE

Spring Framework 6.0 - Baseline

조금 더 자세하고 이해하기 쉬운 글을 찾으신다면!!

https://www.samsungsds.com/kr/insights/java_jakarta.html

Java EE 용어		Jakarta EE 용어	
Java Servlet	javax.servlet	Jakarta Servlet	jakarta.servlet
JavaServer Pages (JSP)	javax.servlet.jsp	Jakarta Server Pages	jakarta.servlet.jsp
JavaServer Faces (JSF)	javax.faces	Jakarta Server Faces	jakarta.faces
Java Message Service (JMS)	javax.jms	Jakarta Messaging	jakarta.jms
Java Persistence API (JPA)	javax.persistence	Jakarta Persistence	jakarta.persistence
Java Transaction API (JTA)	javax.transaction	Jakarta Transaction	jakarta.transaction
Enterprise JavaBeans (EJB)	javax.ejb	Jakarta Enterprise Beans	jakarta.ejb
Java Mail	javax.mail	Jakarta Mail	Jakarta.mail

[표 1] 자바EE와 자카르타EE 용어 비교

https://www.samsungsds.com/kr/insights/java_jakarta.html

Spring Framework 6.0 - Baseline

Jakarta EE: namespace 변경대응 IntelliJ 지원

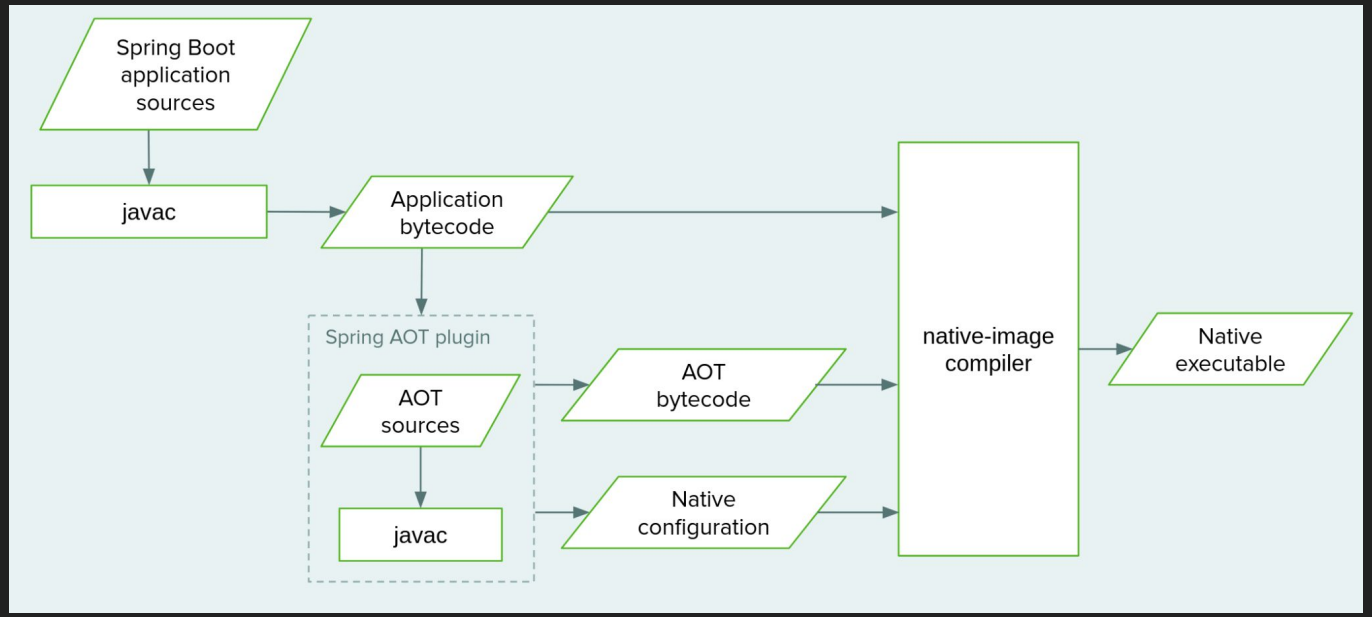
Refactor > Migrate Packages and Classes > Java EE to Jakarta EE

https://www.jetbrains.com/idea/guide/tutorials/migrating-javax-jakarta/

Spring Framework 6.0 - General Core

스프링 애플리케이션을 심층분석하여

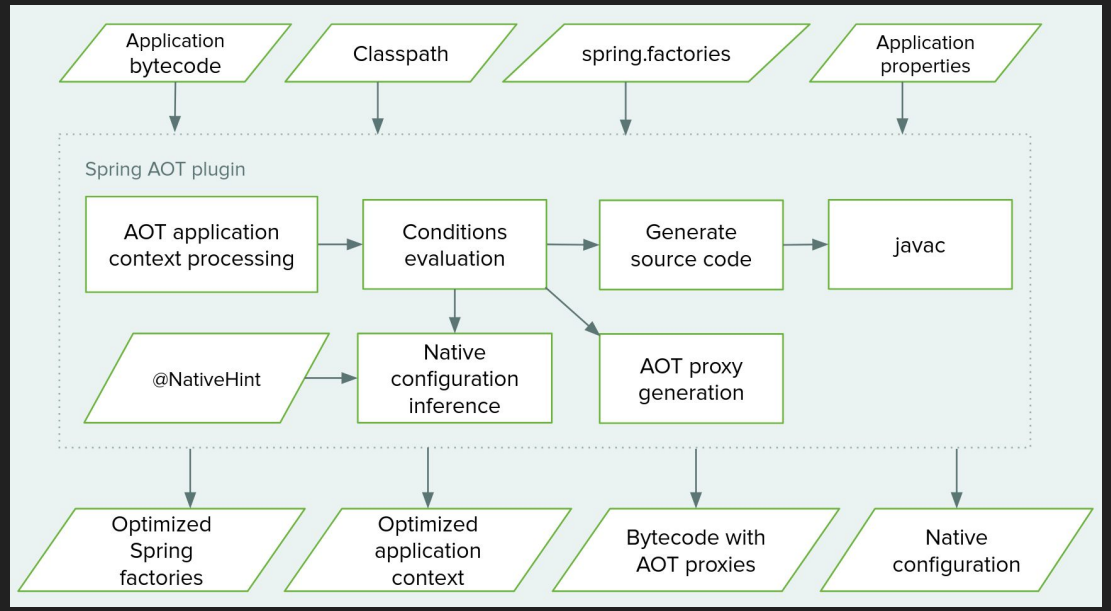
GraalVM에 맞춰 애플리케이션을 변환하고 최적화(Spring Native)



https://spring.io/blog/2021/12/09/new-aot-engine-brings-spring-native-to-the-next-level

Spring Framework 6.0 - General Core

애플리케이션 바이트 코드, 클래스패스, spring.factories, 애플리케이션 속성 을 분석하여 **최적화된 Native 구성을 추론** 한다(그레이들, 메이븐 플러그인 제공).



https://docs.spring.io/spring-native/docs/current/reference/htmlsingle/

Spring Framework 6.0 - General Core

AOT 를 통해 새롭게 Spring Native 도입하면...

- JVM 설치없이 실행이 가능하고
- 실행시 인프라 소모가 줄어들고
- 구동시간을 줄일 수 있고
- 빈 등록과정을 프로그래밍하여 리플렉션을 줄일 수 있다.

~~... 하지만 뭔가 해야할 게 많은 것 같다...~~

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x#general-core-revision

Spring Framework 6.0 - Data Access and Transactions

Jakarta Persistence 3.0.0

Jakarta Persistence 3.0.0 API

The 3.0.0 release is the first release under the `jakarta.persistence.*` namespace.

https://hibernate.org/orm/releases/

Spring Framework 6.0 - Data Access and Transactions

Jakarta Persistence 3.1.0

This release contains following changes:

- EntityManagerFactory and EntityManager interfaces extends java.lang.AutoCloseable interface
- Fixes ClassTransformer.transform to throw Persistence API specific exception
- Adds support for java.util.UUID and GenerationType.UUID
- Adds CEILING, EXP, FLOOR, LN, POWER, ROUND, and SIGN numeric functions to Jakarta Persistence QL and ceiling(), exp(), floor(), ln(), power(), round(), and sign() to Criteria API
- Adds LOCAL DATE, LOCAL DATETIME, and LOCAL TIME functions to Jakarta Persistence QL and corresponding localDate(), localDateTime(), and localTime() to Criteria API
- Adds EXTRACT function to Jakarta Persistence QL
- Adds support for Expressions as conditions in Criteria CASE expressions
- Adds missing definition of single_valued_embeddable_object_field in Jakarta Persistence QL BNF
- Clarifies mixing types of query input parameters
- Clarifies definition of the Basic type
- Clarifies the order of parameters in the LOCATE function
- Clarifies SqlResultSetMapping with multiple EntityResults and conflicting aliases

https://hibernate.org/orm/releases/

Spring Framework 6.0 - Data Access and Transactions

- Hibernate ORM 6.0 (EOL) -> 6.1
  - JPA 2.x -> 3.0 (JPA -> Jakarta Persistence)
  - Jakarta EE 9 지원: javax.persistence -> jakarta.persistence

Compatibility matrix

Hibernate ORM	6.1	5.6
Java	11, 17 or 18	8, 11, 17 or 18
JPA	N/A	2.2
Jakarta Persistence	3.1 and 3.0	3.0

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x#data-access-and-transactions

Spring Framework 6.0 - Data Access and Transactions

- Hibernate ORM 6.0 (EOL) -> 6.1
  - JPA 2.2(끝!) -> Jakarta Persistence 3.0 -> Jakarta Persistence 3.1
  - Jakarta EE 9 지원: javax.persistence -> jakarta.persistence

https://hibernate.org/orm/releases/

Spring Framework 6.0 - General Web

HTTP interface(feign과 유사) - 프록시 생성해서 사용가능...!! —O-)!

```
interface RepositoryService {

    @GetExchange("/repos/{owner}/{repo}")
    Repository getRepository(@PathVariable String owner, @PathVariable String repo);

    // more HTTP exchange methods...

}
```

```
WebClient client = WebClient.builder().baseUrl("https://api.github.com/").build();
HttpServiceProxyFactory factory = HttpServiceProxyFactory.builder(WebClientAdapter.forClient(client)).build();

RepositoryService service = factory.createClient(RepositoryService.class);
```

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x#general-web-revision

Spring Framework 6.0 - General Web

@HttpExchange 를 타입레벨(type level) 이용해서 모든 메서드 제공

```
@HttpExchange(url = "/repos/{owner}/{repo}", accept = "application/vnd.github.v3+json")
interface RepositoryService {

    @GetExchange
    Repository getRepository(@PathVariable String owner, @PathVariable String repo);

    @PatchExchange(contentType = MediaType.APPLICATION_FORM_URLENCODED_VALUE)
    void updateRepository(@PathVariable String owner, @PathVariable String repo,
            @RequestParam String name, @RequestParam String description, @RequestParam String homepage);

}
```

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x#general-web-revision

Spring Framework 6.0 - General Web

RFC 7807 - 'Problem Details for HTTP APIs'

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x#general-web-revision

Spring Framework 6.0 - General Web

Jackson 2.14.1

- https://github.com/FasterXML/jackson/wiki/Jackson-Release-2.14

Jakarta Servlet 6.0

- https://jakarta.ee/specifications/servlet/6.0/

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x#general-web-revision

Spring Framework 6.0 - Spring MVC

PathPatternParser - 후행 슬래시(trailing slash, URL 마지막 /) 허용하지 않음

```java
@RestController
public class MyController {

  @GetMapping("/some/greeting")
  public String greeting {
    return "Hello";
  }

}
```

```java
@Configuration
public class WebConfiguration implements WebMvcConfigurer {

    @Override
    public void configurePathMatch(PathMatchConfigurer configurer) {
      configurer.setUseTrailingSlashMatch(true);
    }

}
```

임시로 이렇게 사용

GET /some/greeting/ != GET /some/greeting

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x#spring-mvc

Spring Framework 6.0 - Spring WebFlux

- PartEvent - Stream multipart form upload
- ResponseEntityExceptionHandler - WebFlux exception
- Flux - non-streaming media types 반환
- Netty 5 기반 Reactor Netty 2 지원
- JDK HttpClient 를 통합한 WebClient

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x#spring-webflux

Spring Framework 6.0 - Observability

Spring Framework 여러 부분에서 Micrometer Observation 사용한 직접계측

- spring-web 은 io.micrometer:micrometer-observation 의존성 가짐

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x#observability

Spring Framework 6.0 - Testing

- JVM 혹은 GraalVM 상에서 AOT 처리된 애플리케이션 컨텍스트 테스트 지원
- HtmlUnit 2.64 요청 파라미터 제어 통합
- Servlet API 6.0 을 기반한 서블릿 Mock(MockHttpServletRequest, MockHttpSession) 지원

https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x#testing

Spring Framework 6.0 - Upgrade

Jakarta EE 9 적용되면서 변경되는 네임스페이스 변경 신경쓰기

- javax.inject -> jakarta.inject
- javax.annotation -> jakarta.annotation
- javax.persistence - > jakarta.persistence
- javax.servlet -> jakarta.servlet

https://github.com/spring-projects/spring-framework/wiki/Upgrading-to-Spring-Framework-6.x

https://github.com/spring-projects/spring-framework/wiki/Upgrading-to-Spring-Framework-6.x#core-container

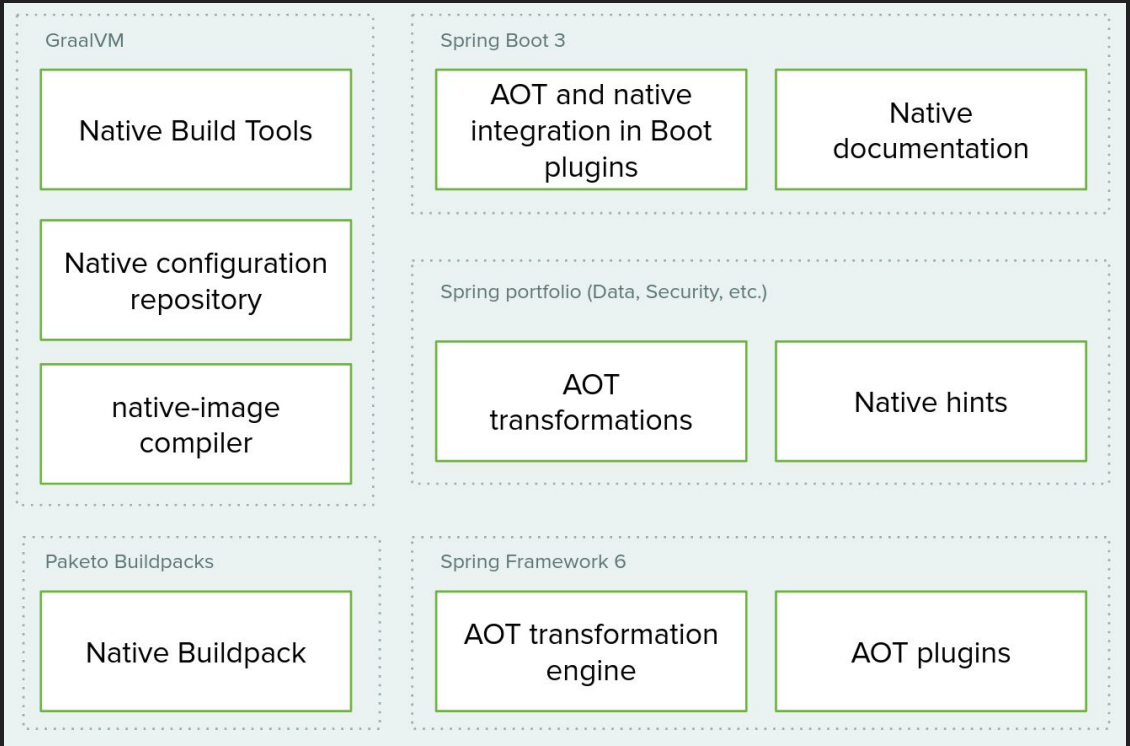
Spring Boot 3

Spring Boot 3.0

- 최소 요구사양 변경
  - Gradle 7.3
  - Java 17
  - Kotlin 1.6
  - Jakarta EE 9
  - Spring Framework 6

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0-Release-Notes

Spring Boot 3.0 - First-class Native Support



https://spring.io/blog/2021/12/09/new-aot-engine-brings-spring-native-to-the-next-level

Spring Boot 3.0.0-M1(2022-01-21)

- Jakarta EE 9 적용
- Jakarta EE 9 미지원 라이브러리 중단 및 제거
  - Apache ActiveMQ
  - Atomikos
  - EhCache 2
  - Hazelcast 3
- CommonsMultipartResolver 지원 중단 및 제거
  - StandardMultipartResolver 기본

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0.0-M1-Release-Notes

Spring Boot 3.0.0-M2(2022-03-25)

- Groovy 4.0
- RxJava 1.x 와 RxJava 2.x 의존성관리 제거 및 RxJava 3 대체
- 이미지 배너 지원 제거(텍스트 배너만 지원)
- Micrometer Observability 지원
- Jakarta EE 9 지원 복귀
  - EhCache 3
  - H2Console
- @ConstructorBinding 기능 향상: 생성자가 2개이상인 경우 선언필요

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0.0-M2-Release-Notes

Spring Boot 3.0.0-M3(2022-05-20)

- Actuator 계측노출 속성 변경
  - old: management.metrics.export.<product> - deprecated
  - new: management.<product>.metrics.export
  - spring-boot-properties-migrator 사용시 알림
- 메인클래스명 탐색 간결화 - Gradle
  - bootJar, bootRun, bootWar 에서 메인 클래스 지정시 발생할 오류 대체
  - springBoot DSL 추가하여 메인클래스 정의
- Spring Security - SAML2 Relying Part 구성 변경
- 내장형 MongoDB 지원 제거됨
- 로깅 일자 형식 변경(ISO-8601) yyyy-MM-dd'T'HH:mm:ss.SSSXXX

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0.0-M3-Release-Notes

Spring Boot 3.0.0-M3(2022-05-20)

- Apache Solr 지원 제거
- Micrometer's JvmInfoMetrics 자동구성
- Jakarta EE 9 지원에 따라 복귀
  - REST Assured(5.0)
  - Pooled JMS(Apache Artemis)
- Micrometer Observability
  - Micrometer Observation API 자동구성
  - Micrometer Tracing 자동구성
  - Micrometer OtlpMeterRegistory
- Apache Kafaka 비동기 Acks 활성화
  - spring.kafka.listener.async-acks: true
  - spring.kafka.listener.async-mode: manual | immediate

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0.0-M3-Release-Notes

Spring Boot 3.0.0-M4(2022-07-21)

- **후행 슬래시(trailing slash) 매칭 구성 비활성화**
  - GET /some/greeting != GET /some/greeting/
- **Hibernate ORM 6.1**
  - **spring-boot-starter-data-jpa 내에 Hibernate Group ID Change**
  - org.hibernate:hibernate-core -> org.hibernate.orm:hibernate-core
- Flyway 9.0 지원
- R2DBC 1.0 지원
- Elasticsearch Clients Templates
- Reactor Netty 가 없으면 JdkClientHttpConnector 가 자동구성되어 WebClient 이 사용함

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0.0-M4-Release-Notes

Spring Boot 3.0.0-M5(2022-09-23)

- spring.data 접두사 - Spring Data 전용 확정
  - Cassandra: spring.data.cassandra.* -> spring.cassandra.*
  - Redis: spring.redis.* -> spring.data.redis.*
- JMX 종단점(Endpoint) 노출: health 만 노출
- 액츄에이터 종단점 민감데이터 기본 마스킹 처리 및 속성 설정
- Flyway 구성 사용자정의 지원
- @EnableBatchProcessing 필요없음 자동인식됨
- MongoDB Health Check 제공
- spring.factories 를 이용한 자동구성 등록지원 기능 제거
- Spring Native 지원
- @SpringBootTest 에서 메인메서드 사용여부 결정 가능

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0.0-M5-Release-Notes

Spring Boot 3.0.0-RC1(2022-10-21)

- server.max-http-header-size: 웹컨테이너 일괄적용
- Graceful Shutdown 에 사용되는 SmartLifeCycle 페이즈 변경
- 스프링 부트 태스크 Properties 이용해서 정의 가능
- build-info.properties 항목 중 속성 제외가능
- Spring Native -> GraalVM Native
- Spring Data JDBC 자동구성 지원

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0.0-RC1-Release-Notes

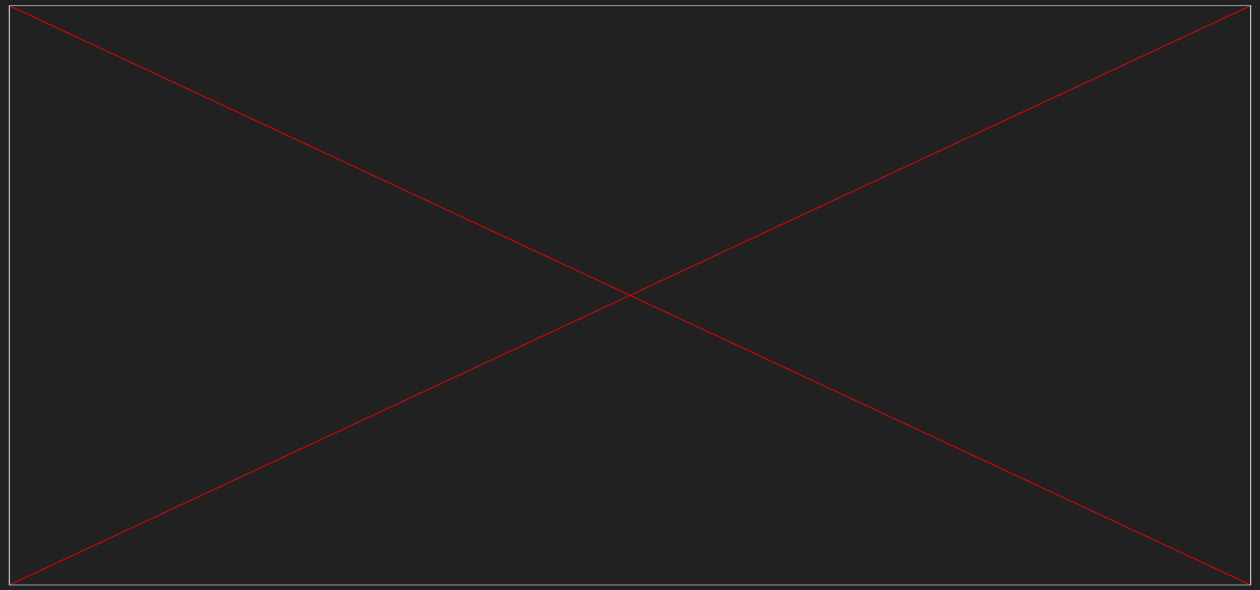
Spring Boot 3.0.0-RC2(2022-11-11)

- Jakarta EE 10 지원
- Jetty 사용 시 Servlet 지원여부 확인하여 조정
- httptrace 종단명 변경: httpexchange
- 액츄에이터 종단점 별로 ObjectMapper 사용 가능
- Spring Native 사용시 Graal 22.3 혹은 최신

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0.0-RC2-Release-Notes

Spring Boot 3.0.0 GA(2022-11-24)

- 스프링 부트 2.0 출시 후 4.5년!
- Spring Framework 6 과 GraalVM 지원(정식버전은...)



https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0-Release-Notes

정리

Java

- Java 11을 넘어서 17까지!
- 점점 제공되는 기능이 풍부해지고 있음
- 코틀린...? 굳이 자바에서 코틀린으로 옮겨갈 필요성이 줄어들고 있음

정리

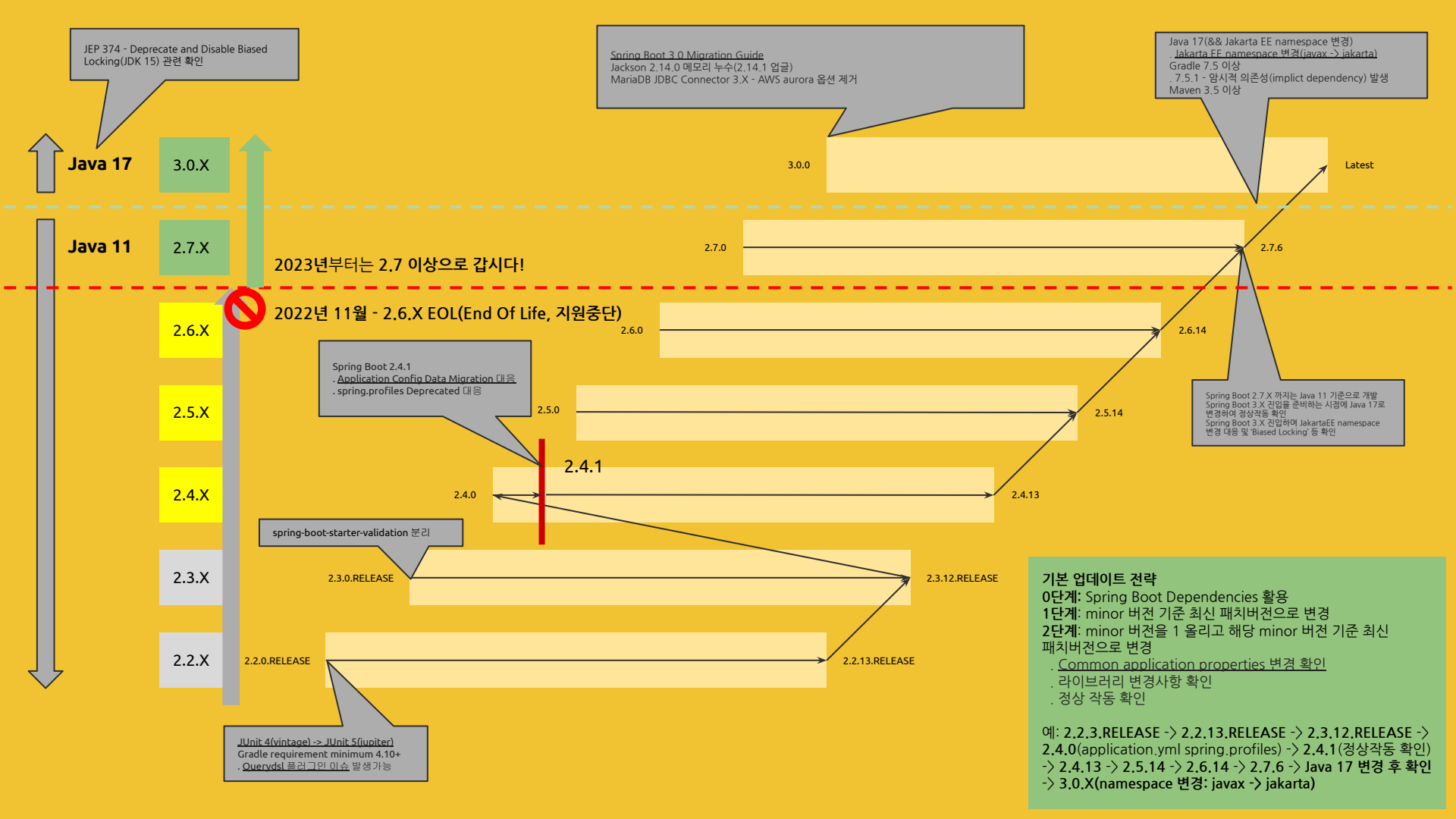
Spring Framework 6.0

- Java EE 8 -> Jakarta EE 9 변경
- 네임스페이스(namespace) 변경: **javax.* -> jakarta.***
- AOT(Ahead-Of-Time) 엔진 도입
  - Spring Native 이미지(Graal VM image) 빌드
  - 빌드 단계에서 정적분석을 통해서 최적화 작업 수행
- Micrometer 관측 지원 강화
- 실험실에서 연구되던 기능들 반영
  - Spring Native(GraalVM)

정리

Spring Boot 3.0

- Java 17과 Kotlin 1.6 이상 지원
- Spring Framework 6.0 적용
  - Jakarta EE 9 최적화
- 그레이들 7.X(7.5 이상)
- 미사용 라이브러리 및 지원 중단


JEP 374 - Deprecate and Disable Biased Locking(JDK 15) 관련 확인
Spring Boot 3.0 Migration Guide
Jackson 2.14.0 메모리 누수(2.14.1 업글)
MariaDB JDBC Connector 3.X - AWS aurora 옵션 제거
Java 17(&& Jakarta EE namespace 변경)
. Jakarta EE namespace 변경(javax -> jakarta)
Gradle 7.5 이상
. 7.5.1 - 암시적 의존성(implict dependency) 발생
Maven 3.5 이상
Java 17
3.0.X
3.0.0
Latest
Java 11
2.7.X
2.7.0
2.7.6
2023년부터는 2.7 이상으로 갑시다!
2.6.X
2022년 11월 - 2.6.X EOL(End Of Life, 지원중단)
2.6.0
2.6.14
Spring Boot 2.4.1
. Application Config Data Migration 대응
. spring.profiles Deprecated 대응
2.5.X
2.5.0
2.5.14
Spring Boot 2.7.X 까지는 Java 11 기준으로 개발
Spring Boot 3.X 진입을 준비하는 시점에 Java 17로 변경하여 정상작동 확인
Spring Boot 3.X 진입하며 JakartaEE namespace 변경 대응 및 'Biased Locking' 등 확인
2.4.1
2.4.X
2.4.0
2.4.13
spring-boot-starter-validation 분리
2.3.X
2.3.0.RELEASE
2.3.12.RELEASE
2.2.X
2.2.0.RELEASE
2.2.13.RELEASE
JUnit 4(vintage) -> JUnit 5(jupiter)
Gradle requirement minimum 4.10+
. Querydsl 플러그인 이슈 발생가능
기본 업데이트 전략
0단계: Spring Boot Dependencies 활용
1단계: minor 버전 기준 최신 패치버전으로 변경
2단계: minor 버전을 1 올리고 해당 minor 버전 기준 최신 패치버전으로 변경
. Common application properties 변경 확인
. 라이브러리 변경사항 확인
. 정상 작동 확인
예: 2.2.3.RELEASE -> 2.2.13.RELEASE -> 2.3.12.RELEASE -> 2.4.0(application.yml spring.profiles) -> 2.4.1(정상작동 확인) -> 2.4.13 -> 2.5.14 -> 2.6.14 -> 2.7.6 -> Java 17 변경 후 확인 -> 3.0.X(namespace 변경: javax -> jakarta)

참고

Java

- https://docs.oracle.com/en/java/javase/17/index.html

Jakarta

- https://jakarta.ee/release/10/
- https://eclipse-ee4j.github.io/jakartaee-platform/jakartaee9/JakartaEE9ReleasePlan
- https://eclipse-ee4j.github.io/jakartaee-platform/jakartaee10/JakartaEE10ReleasePlan

Jakarta Persistence & Hibernate

- https://hibernate.org/orm/releases/
- https://github.com/jakartaee/persistence/releases/tag/3.1-3.1.0-RELEASE

https://newsroom.eclipse.org/eclipse-newsletter/2022/march/what%E2%80%99s-new-jakarta-persistence-31

참고

Spring

- https://github.com/spring-projects/spring-framework/releases/tag/v6.0.0
  - https://github.com/spring-projects/spring-framework/wiki/What%27s-New-in-Spring-Framework-6.x/
  - https://github.com/spring-projects/spring-framework/wiki/Upgrading-to-Spring-Framework-6.x
- https://www.youtube.com/watch?v=aUm5WZjh8RA
- https://www.springcloud.io/post/2022-02/springframework-6/#gsc.tab=0
- https://www.jetbrains.com/idea/guide/tutorials/migrating-javax-jakarta/
- https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0-Release-Notes

Spring Native

- https://spring.io/blog/2021/12/09/new-aot-engine-brings-spring-native-to-the-next-level
- https://docs.spring.io/spring-native/docs/current/reference/htmlsingle/

끗! See you again!

궁금하신 것은 ihoneymon@gmail.com 으로 문의주세요.